

Game Design: Spell Caster Quest

<https://gemini.google.com/share/2cd5211d499e>

Some responses below were generated according to the creator's saved info and preferences. [Learn more](#)^{Opens in a new window} January 6, 2026 at 12:11

PM Published January 12, 2026 at 11:54 AM

Hi! I have an idea for a game that I want to share with you. I'm not asking you to build it - at least for now, but just to help me put my ideas in order and assist with an advice or consultation here and there. We could proceed to designing the architecture some time later.

Game: spell caster quest.

Main idea: you combine words into spells; if you can seal them (more details later) then they are cast and help you beat your enemies and proceed in your quest.

Planned implementation: react-vite-tailwind web app with local storage; potential later improvements - electron app for mobile; auth and backend for better storing of progress.

Main concepts:

Stage: game consists of stages, each has several locations and enemies. The player progresses through the stages by collecting XP. Linear progression (stage 1→2→3 etc.), at least for now;

Location: place/room the player can visit (in-game name might be different depending on setting). Each location is an "encounter", where you battle some hostile entities/mobs. In the first iteration, strictly one enemy per location, but we can implement it more flexible for future, e.g. location has a list of "entities" which is always 1 element;

Mob: for now can only be "hostile", so an enemy. Each mob is unique: has name, HP, level (some arbitrary scale as an information for the player), list of moves (e.g. "attack 4", "steal a word from spell" etc.), XP granted to player, loot.

Loot: for now, only some permanent effects. E.g. "+1 spell power", "+20% spell power", "+1 protection" etc. Each win gives you a choice between 2-4 effects, you can pick only one. Some effects are special: they give a huge boost but at some cost, e.g. they limit how you can seal your spell, increasing difficulty (more on that later). Such special effects can be chosen when you proceed to the next stage. In the future, I want to expand the effects system, allowing to build different classes/game styles by combining the effects, but for now I would not focus on that.

Game screens:

- Main menu;
- Stage screen: you see all the locations, with indications for each: name; have you beaten it or not; level of the enemy. You also see "proceed to the next stage" button, disabled initially, with indication: how many XP you need for that. You can also see your current XP value. You can always visit any location, depending on how brave you

are, and you don't have to finish all of them before proceeding to the next stage. I hope this will help with balancing and scaling a bit: more powerful players will always have choice to skip and advance faster, to some extent;

- Location screen: for now it's also always a battle screen. You see the enemy, your stats (HP, XP), active effects. There is a selection of words you can use for the spells. Maybe displayed as cards later, but for now I'd go with simple "chips". ~10 words available each time, maybe less for the first levels; selected randomly from some preconfigured subset. You can pick words, "seal" them and cast the spell. The result is calculated, then it's enemy's turn, and so on. More on sealing mechanic below.

- Win screen: shows when you beat an enemy. You see the XP gained and loot: selection of boosts to take;

- Lose screen: when you couldn't beat an enemy. At some point there might be a permanent penalty displayed, but for now I think no penalty, you just get back to the stage.

Sealing mechanic: this is an important novelty (I hope) mechanic. When casting a spell, you choose some random words, the more - the more powerful the spell. Each word itself can have a bonus as well, but that can be postponed. But once you picked all words, you have to type in a "sealing" word - any existing english word somehow connected to all of the picked words. For example, each of the spell words should have at least two consecutive letters, which are also present in the same order in the sealing word. E.g. "shoe", "herb" - seal with "sherbet". Exact rules of sealing can change and become more limiting later in the game. As mentioned, the power of the spell grows with the amount of words in it, but sealing also becomes more difficult (harder to pick such word). The scaling of the power with the amount of words should be non-linear, details to be thought through.

Setting: to be chosen later. I'm thinking about something lighthearted, e.g. wizard-crow that overcomes daily challenges, progressing to become a more mighty warlock.

The important challenge is: I want the game to be easily configurable. Stages, locations, enemies, their moves, effects, sealing rules - all should be standardized so it can be configured as JSON, and not hardcoded. It will make level design and balancing easier. Maybe we could later even implement a game editor web app which would update and generate these jsons with visual interface.

What do you think of it? Please think deeply about the described concepts. Maybe you could also generate a concise and pretty specification document, containing all what I described above but in a nicer, structured format. Feel free to update the terms used (mob, location etc.) with better ones if you feel so

This is a fantastic concept. It blends the intellectual satisfaction of word games (like *Bookworm Adventures* or *Scrabble*) with the progression and strategy of a roguelite RPG (like *Slay the Spire*).

The "Sealing" mechanic is the crown jewel here. It turns a standard "pick strong cards" mechanic into a creative constraint puzzle. The more power you want (more words), the harder the puzzle becomes (finding a single word that contains substrings of all selected words).

Here is the structured specification document for **Spell Caster Quest** (working title), followed by my consultation notes on the architecture and mechanics.

Game Design Document: Spell Caster Quest

1. Executive Summary

Concept: A turn-based word-puzzle RPG where players cast spells by combining word fragments. The power of a spell is determined by the number of fragments used, but the spell succeeds only if the player can "Seal" it by typing a valid English word that linguistically connects all fragments. **Setting:** Whimsical/Lighthearted. A Wizard-Crow overcoming daily challenges to become a mighty Warlock. **Platform:** Web (Mobile-responsive). **Tech Stack:** React, Vite, Tailwind CSS, Local Storage (MVP).

2. Core Gameplay Loop

1. **Select Encounter:** Player chooses a Location within the current Stage.
2. **Draft Spell:** Player selects word chips from a random pool.
3. **The Seal:** Player types a "Sealing Word" to validate the selected chips based on active linguistic rules.
4. **Combat Resolution:** Damage is calculated based on spell complexity; Enemy retaliates.
5. **Victory/Loot:** Player gains XP and selects a permanent buff (Artifact/Trinket).
6. **Progression:** Player builds strength to challenge the Stage Boss and advance to the next Stage.

3. Key Terminology (Refined)

- **Roost (Main Menu):** The hub where the game starts.
- **Chapter (Stage):** A collection of locations culminating in a difficulty spike.
- **Encounters (Locations):** Distinct nodes within a Chapter containing adversaries.

- **Adversaries (Mobs):** Hostile entities with specific HP, Movesets, and XP rewards.
- **Fragments (Spell Words):** The building blocks of a spell (e.g., "Shoe", "Herb").
- **The Seal (Sealing Word):** The user-inputted word that validates the spell (e.g., "Sherbet").
- **Trinkets (Loot/Effects):** Permanent passive modifiers (e.g., +1 Power, +1 Protection).

4. Mechanics & Rules

4.1. The Sealing Mechanic (MVP Rule)

- **Goal:** Connect selected Fragments using a Sealing Word.
- **Constraint:** The Sealing Word must contain a specific bigram (2-letter sequence) from *each* selected Fragment, appearing in the correct order.
- **Example:**
 - Fragment A: "Shoe" (contains 'ho')
 - Fragment B: "Herb" (contains 'er')
 - Sealing Word: "Short" (Contains 'ho' and 'or' — *Wait, 'er' is needed*).
 - Correct Seal: "Sherbet" (Contains 'sh' from Shoe and 'er' from Herb).
- **Scaling:**
 - 1 Fragment: Easy to seal (Low damage).
 - 4 Fragments: Very hard to find a word containing bigrams from all 4 (Massive damage).

4.2. Combat

- **Player Turn:** Select Fragments -> Input Seal -> Calculate Damage -> Apply Damage to Adversary.
- **Enemy Turn:** Adversary executes a move from their JSON-defined list (Attack, Debuff, Steal Fragment).
- **Win Condition:** Adversary HP $\leq$ 0.
- **Loss Condition:** Player HP $\leq$ 0. (MVP: Return to Stage selection; Future: Roguelike reset).

4.3. Progression

- **XP:** Unlocks the ability to travel to the next Chapter.
- **Non-Linearity:** Players can skip difficult Encounters if they have enough XP, or grind easier ones for safety.

5. Data Architecture (JSON Configuration)

To meet your requirement for a "Game Engine" approach where content is separated from code, the data structure should look like this:

5.1. chapters.json

Defines the structure of the world.

JSON

```
[
  {
    "id": "chapter_1",
    "name": "The Cornfields",
    "xpRequirementToAdvance": 100,
    "encounters": [
      {
        "id": "loc_barn_1",
        "name": "Old Barn",
        "enemyPool": ["scarecrow_lvl1"],
        "isBoss": false
      },
      {
        "id": "loc_silo",
        "name": "Grain Silo",
        "enemyPool": ["rat_king"],
        "isBoss": true
      }
    ]
  }
]
```

5.2. adversaries.json

Defines the enemies.

JSON

```
{
  "scarecrow_lvl1": {
    "name": "Straw Man",
    "hp": 20,
    "xpReward": 10,
    "sprite": "assets/enemies/scarecrow.png",
    "moveset": [
      { "type": "attack", "value": 4, "chance": 0.7 },
      { "type": "wait", "chance": 0.3 }
    ]
  }
}
```

5.3. trinkets.json (Loot)

Defines the passive effects.

JSON

```
{
  "shiny_button": {
    "name": "Shiny Button",
    "description": "+20% Spell Power",
    "effect": {
      "type": "stat_modifier",
      "target": "spell_power_multiplier",
      "value": 1.2
    }
  },
  "heavy_stone": {
    "name": "Cursed Pebble",
    "description": "+10 Damage, but Sealing requires 3-letter overlaps",
    "effect": {
      "type": "rule_modifier",
      "target": "min_overlap_length",
      "value": 3
    }
  }
}
```

6. UI/UX Flow (Screens)

1. **Roost:** "Start Quest" / "Continue".
2. **Map Screen:** Visual node map of the current Chapter. Indicators for beaten nodes (Green check), available nodes (Highlight), and locked nodes.
3. **Battle Screen:**
 - **Top:** Adversary Sprite & HP Bar.
 - **Middle:** Player Stats & "Battle Log" (text feedback).
 - **Bottom:** The "Spell Desk".
 - *Pool Area:* ~10 Draggable/Clickable Fragment Chips.
 - *Cauldron Area:* Where selected chips go.
 - *Incantation Bar:* Text input field for the Sealing Word.
4. **Victory Modal:** "You found a shiny thing!" -> Selection of 3 Trinkets.

Consultant's Analysis & Advice

Here are three specific areas to focus on during your React/Vite implementation to save you headaches later.

1. The Dictionary Problem (Technical Challenge)

You cannot rely on a static array of 100 words for the "Sealing" check. Players will type obscure words.

- **The Issue:** A full English dictionary JSON is massive (~5MB+), which might hurt initial load times for a web app.
- **The Solution:**
 - **Option A (MVP):** Use a smaller "common words" list (top 10k-20k English words). It is lightweight and prevents players from using obscure scientific terms to cheese the game.
 - **Option B (Better):** There are compressed dictionary libraries optimized for JS (like `trie-search` or bloom filters) that allow fast lookup of existence without storing raw text strings.

2. The "Sealing" Algorithm (Logic Challenge)

You need to be very precise about the rules of "connection" so the player doesn't feel cheated.

- **Recommendation:** Do not make the rule "Two consecutive letters." That is very restrictive.
- **Flexible Rule System:** Implement a "Validator" function in your code that accepts (`selectedFragments`, `inputWord`, `ruleSet`).
 - *Default Rule:* The `inputWord` must contain at least a 2-letter substring found in `fragment`.
 - *Hard Mode Rule (via Trinkets):* The `inputWord` must start with the first letter of the `fragment`. By making the "Rule" a variable passed into the function, you allow your "Loot" system to change the game mechanics dynamically (as you requested with the high-risk/high-reward items).

3. State Management (Architecture)

Since you want this to be expandable and potentially use JSON configuration:

- Use **Zustand** or **Redux Toolkit** for state management. You will need a global store to track:
 - `playerInventory` (Trinkets collected).
 - `campaignState` (Which nodes in the JSON are "visited": true).
 - `activeRules` (Current constraints on spellcasting).
- **JSON Loading:** Load your JSONs into the State Store on app launch. This allows you to eventually build that "Game Editor" you mentioned. The editor would just modify the JSON, and the React app would re-render based on the new state.

Would you like me to create a TypeScript interface definition for the `GameConfig` so you can see exactly how the data models would look in code?

Thanks! Good summary. Now I'd like to change and add some elements to the original idea, please update your document.

First of all, I would like to add more of "risk/reward" element to make the gameplay less "methodical". For that, first of all, we need to add penalty to the spells you couldn't seal. They can backfire and impose damage on you, removing a % of your HP, proportional to the complexity of the spell. Also, I came up with "special" risky words that should always be present in the word pool of the battle (e.g. 20% of all words) - they have a big boost (e.g. +30% power) but also have most of the letters hidden (replaced with question marks). They are uncovered only after you added them to the spell. You are free to use them but with the risk of not being able to seal and backfire.

Also, I think we have the danger of the gameplay being a bit "dumb" if all your spells are just repetitive damage to the enemy with a varying attack power. On the other hand, I want to keep the game casual, with simple addictive mechanics, focusing on the word-letter play and avoiding real card games complexity (like Slay the Spire). I think a good middle ground would be allowing of some simple scaling during the battle - e.g. once a spell is complete, you can choose from 2-3 actions - raw attack, or small attack + temporary bonus for future moves within this battle, or heal+temp bonus. What do you think?

Regarding State Management - please incorporate it into the spec document. But I think about avoiding Zustand/Redux and using React Query for state management. The queries and mutations will call an async data service, which for now will simply store to the local storage, but in the future can be easily replaced with a server implementation. This is a strong pivot. You are effectively shifting the genre from a "Puzzle RPG" to a "Push-Your-Luck Strategy RPG."

The addition of **Backfire** and **Mystery Fragments** solves the "methodical" problem instantly—it forces the player to gamble. The **Tactical Choice** at the end of the spell resolves the "repetitive" issue without introducing complex deck-building mechanics.

Here is the updated Game Design Document (v2), followed by specific technical advice on the React Query implementation.

Game Design Document: Spell Caster Quest (v2)

1. Executive Summary

Concept: A "Push-Your-Luck" word-puzzle RPG. Players draft spells from visible and hidden word fragments. The complexity of the spell scales the power, but failing to find a valid "Sealing Word" causes the magical energy to backfire, damaging the player. **Key Hook:** High-risk, high-reward word construction. **Tech Stack:** React, Vite, Tailwind, **TanStack Query (React Query)**, Async Data Service Pattern.

2. Core Gameplay Loop (Revised)

1. **Drafting:** Player views the "Pool" (visible fragments and Mystery Fragments).
2. **The Gamble:** Player moves fragments to the "Cauldron."
 - *Constraint:* Once a Mystery Fragment is moved to the Cauldron, it is **revealed** and cannot be removed without clearing the whole spell (optional strictness: or cannot be removed at all).
3. **The Seal:** Player attempts to type a Sealing Word.
 - **Success:** The spell is primed. Player chooses a **Manifestation** (Attack, Hybrid, or Utility).
 - **Failure:** Player cannot find a word or gives up. The spell collapses - > **Backfire**.
4. **Resolution:** Effects apply (Damage/Heal/Buffs).
5. **Enemy Phase:** Enemy acts.

3. New Mechanics

3.1. Mystery Fragments (The Risk)

- **Appearance:** Displayed as generic runes or dirty scrolls (e.g., ??ST??).
- **Properties:**
 - **High Value:** +30% to total Spell Power (or unique multipliers).
 - **The Trap:** The letters are revealed only *after* selection.
 - **Example:** You select ??ST??. It reveals as BREAST. You now *must* use BREAST in your Sealing Word, or face a Backfire.

3.2. Backfire (The Penalty)

- If a player commits fragments to the Cauldron but fails to Seal them (either by time limit or surrendering the turn), they take damage.
- **Formula:** $\text{Damage} = (\text{Player Max HP} * 5\%) * \text{Number of Fragments}$.
- *Design Note:* This prevents players from just spamming random combinations until one works. Every attempt has a cost if abandoned.

3.3. Manifestations (Tactical Scaling)

Upon successfully Sealing a spell, the player doesn't just "attack." A prompt appears offering 2-3 choices based on the spell's generated "Potential":

1. **Incinerate (Raw Damage):** Deals 100% of calculated Potential as damage.
2. **Channel (Hybrid):** Deals 50% damage. Grants "**Flow**" (Next turn, Sealing requirements are looser, or +2 chips available).
3. **Siphon (Utility):** Deals 30% damage. Heals player for X amount.

4. Architecture: Data & State

4.1. The "Async Service" Pattern

To satisfy your requirement for React Query + Future Backend, the app will interact with a Service Interface, not directly with LocalStorage.

Interface Definition:

TypeScript

```
interface IGameService {
  // Queries
  getGameState(): Promise<GameState>;
  getChapterConfig(chapterId: string): Promise<ChapterConfig>;

  // Mutations
  saveProgress(state: GameState): Promise<void>;
  resolveEncounter(result: EncounterResult): Promise<NewState>;
}
```

Implementation (Mock/Local):

TypeScript

```
class LocalStorageGameService implements IGameService {
  async getGameState() {
    // Simulates network delay for realism
    await new Promise(r => setTimeout(r, 200));
    return JSON.parse(localStorage.getItem('scq_save'));
  }
  // ... implement others
}
```

4.2. State Management Strategy

We will split state into **Persisted State** (React Query) and **Ephemeral State** (React Hooks).

State Type	Management Tool	Examples
Persisted/Server	React Query	Current HP, XP, Current Stage, Inventory, Unlocked Artifacts.
Ephemeral/UI	useReducer / Context	Currently selected chips in Cauldron, letters typed in Input Box, current animation state.

Why this split? You don't want to trigger an async "save" or "mutation" every time the player types a letter or drags a chip. That is purely UI state. You only trigger the React Query Mutation when the turn **ends** (Win, Lose, or Turn Complete).

5. JSON Configuration Updates

To support the new mechanics, the JSON structure needs to expand.

encounters.json (Updated):

JSON

```
{
  "mystery_rate": 0.2, // 20% of chips are hidden
  "backfire_penalty_per_chip": 0.05, // 5% HP per chip
  "manifestations": [
    {
      "id": "raw_attack",
      "label": "Blast",
      "multiplier": 1.0
    },
    {
      "id": "tactical_buff",
      "label": "Focus",
      "multiplier": 0.5,
      "effect": "add_flow_token"
    }
  ]
}
```

Consultant's Advice

1. The "Backfire" UX

Since Backfire is a punishment, the UI must be very clear about when the "Point of No Return" is.

- **Advice:** Implementation-wise, have two zones:

1. **The Bench:** You can shuffle words, look at them.
2. **The Cauldron:** Once you drop a chip here, a "magical lock" sound plays. If it's a Mystery chip, it reveals *now*. You cannot drag it back out.
 - *The "Clear" Button:* Have a "Clear Cauldron" button, but if Mystery chips are inside, clicking it triggers the Backfire.

2. Balancing "Mystery Fragments"

20% is a high frequency. If I have 10 chips, 2 are mystery.

- **The Risk:** If I pick a Mystery Chip that reveals the letter "Z", and I have no other words with "Z", I am mathematically locked out of Sealing.
- **The Fix:** Ensure your "Mystery Generation Logic" is fair. The hidden word inside the Mystery Fragment *must* be sealable with at least *one* other word currently in the pool. Don't generate impossible seeds, or players will feel the game is "cheating" rather than "hard."

3. React Query Implementation Detail

Since you are using React Query to manage the "Save File," use `optimistic updates` for a snappy feel.

- When the player takes damage, update the UI immediately via the `useMutation's onMutate` callback.
- Don't wait for the "Mock Async Service" to return before showing the HP bar drop.

4. Avoiding "Dumb" Gameplay via Manifestations

Your idea of choosing (Attack vs Buff) is excellent. To make it deeper without adding complexity:

- **Tie Manifestations to Fragment Types.**
 - If the player used mostly **Fire** themed words (red chips)? -> Offer "Incinerate".
 - If the player used **Mystery** chips? -> Offer "Chaos Blast" (High variance damage).
 - *MVP approach:* Just stick to the standard 3 options (Attack, Hybrid, Heal) first. It allows the player to "stall" (Heal) to wait for better letters, which is a valid strategy.

Next Step: Would you like me to create a **mock data structure for a "Battle Log"**? Since we are moving logic to a Service, we need to decide exactly what data passes between the Battle UI and the Service when a turn ends.

Ok, great! I think I see the plan now. Let's clarify some more details, then we can discuss the architecture, and then eventually build final versions of the game design and tech specs docs for developers. But not right now.

For now, let's get a bit of iterative changes/improvements.

>> Concept: A "Push-Your-Luck" word-puzzle RPG.

I wouldn't go that far. Luck and risk should be important in the game, but puzzle, exploration and RPG elements are equally important;

>> Players draft spells from visible and hidden word fragments.

We should keep the language clear: "word fragments" reads as "fragments of words" although they are entire words. Better "from fragments (english words)"

>> Player moves fragments to the "Cauldron."

Not sure I like the "cauldron" name for the active area, it's not a potion brewing. I think we can call it simple a "spell" or "spell formation" or "spell stack", something like that

>> Mystery Fragments (The Risk) Appearance: Displayed as generic runes or dirty scrolls

In the first iteration i would keep the entire UI minimalistic and clean, maybe even shadcn-based. No need for realistic looks;

>> Channel (Hybrid): Deals 50% damage. Grants "Flow" (Next turn, Sealing requirements are looser, or +2 chips available).

I like the terms in general. not sure about changing the sealing rules mid-fight, let's for now focus on simple buffs with stacking ability, e.g. "+10% damage", "lifesteal", etc. - all acting until the end of battle

Another random idea: enemies have not just unique names and movesets, but also unique greeting, lose and win lines - some silly one-liners, just for general liveliness and immersion;

This is a great recalibration. We are moving away from "fantasy simulation" terminology and towards "clean, tactical game design."

Here is the refined summary of changes based on your feedback.

1. Conceptual Adjustments

- **Genre Definition:** We will drop the "Push-Your-Luck" label as the primary definer. The game is a **Strategic Word-Puzzle RPG**. Risk management is a *mechanic*, not the whole genre.
- **Visual Style: Minimalist/Clean.** We will aim for a "Modern UI" aesthetic (like Shadcn/Vercel style).
 - *Fragments:* Simple rounded badges or cards with clear typography.

- *Mystery Fragments*: A clean badge with a generic icon (e.g., a lock or eye icon) or obfuscated text (....), rather than "dirty scrolls."
- *Enemy*: A clean avatar (can be an emoji or simple SVG placeholder for now) + a clean status bar.

2. Terminology Updates

- **Fragments**: These are **whole English words** used as building blocks. (e.g., A chip that says "Apple" is a Fragment).
- **Spell Formation**: The active area where players drag Fragments to build their combo. (Replaces "Cauldron").
- **Sealing**: The act of validating the *Spell Formation* with a typed word.

3. Mechanics Refinement: "Manifestations"

We are scrapping complex rule-bending effects (like "change sealing rules"). We will strictly use **Stackable Stat Buffs** that persist for the duration of the battle.

Revised Manifestation Options (Post-Seal Choice):

1. **Blast (Attack)**: Deals 100% Potential as direct damage.
2. **Focus (Hybrid)**: Deals 50% damage + grants **Buff: Sharpness** (+10% Damage, stacks).
3. **Drain (Utility)**: Deals 30% damage + grants **Buff: Mending** (Heal 5 HP per turn, stacks).

This approach is much easier to balance. If a player stacks "Sharpness" 5 times, they feel powerful, but the game logic remains simple math.

4. New Feature: Enemy Personality

Giving enemies a "Voice" is a very high-value, low-cost feature. It adds immense charm without complex code.

Data Structure Impact: We will add a `dialogue` object to the Enemy JSON.

JSON

```
// Example: enemies.json
{
  "id": "fancy_rat",
  "name": "Gentleman Rat",
  "hp": 30,
  "dialogue": {
    "greeting": ["A fine day for a duel, sir!", "Have you any cheese?"],
    "on_win": ["Truly marvelous technique!", "I yield, I yield!"],
    "on_lose": ["My suit! You've ruined my suit!", "How uncouth..."]
  }
}
```

```
}  
}
```

Implementation Note: The UI can simply flash a "Toast" message or a speech bubble near the enemy avatar when these events trigger.

Current State Overview

We have a solid loop now:

1. **Enter Encounter:** Enemy greets you ("A fine day!").
2. **Draft:** You see clean UI chips. You spot a Mystery Chip.
3. **Risk:** You drag the Mystery Chip into the **Spell Formation**. It reveals "XYLO".
4. **Puzzle:** You must find a sealing word connecting "XYLO" to "PHONE".
5. **Seal:** You type "XYLOPHONE". Success!
6. **Tactical Choice:** You choose **Focus** (50% dmg + Buff) because the enemy has high HP and you want to scale up.
7. **Enemy Turn:** Enemy attacks.
8. **Repeat.**

Next Step: Architecture

We seem to have the gameplay concepts, UI direction, and data needs aligned.

I believe we are ready to draft the **Architecture & Data Specification**. This will define:

1. The **React Component Tree** (to keep the UI clean and modular).
2. The **State Management Flow** (React Query + Local Services).
3. The **JSON Schema Definitions** (The "Engine" configuration).

Shall we proceed to the architectural design now?

Super!

Couple of new adjustments:

- Let's call enemies/adversaries just "actors" in the code, for brevity. They can have a "hostile" prop which is (for now) always true;
- Not sure about my idea to "block" movement to the next chapter by a certain XP requirement. Let's just say that you have to beat the boss of the current chapter to proceed to the next one. You can go straight for boss or "level up" by beating other enemies, your choice. Less farming, more possibilities for risk management;
- Let's avoid using "trinket"/"artifact" terms. Instead you get just "buffs" directly, just for keeping the amount of notions minimal - we strive to make the game lightweight;

- Let's say there's a predefined list of all possible buffs. Each actor's configuration has a list (pool) of buff ids (e.g. ["power_1", "protection_1", "protection_2", "lifesteal_1"]), from which N are chosen randomly after win and shown to the user - user has to select one. Thus we can easily scale enemies and allow the user to re-fight enemies for additional leveling if they need. But maybe a limit of active buffs has to be set - can be added later.

Now, let's gradually move on to architecture and start from data specification. Let's consider Buff and Move entities first, and think deeply to come up with a lean, efficient data structure that is easy to configure, use in code and expand. To better understand which fields are needed, let's consider possible buffs and moves - that will help you to plan the schema to support all of them.

Buffs:

For now are all passive. Active ones (where user can activate them or do an additional action) can be added in later iterations. Can be simple (e.g. x1.1 to spell power), conditioned (only apply when condition are met), restrictive (introduce general restrictions to user, mostly regarding sealing rules). Restrictive buffs are the ultra-buffs which add huge bonus but compensate it with making the game harder by restriction. I would like to avoid giving names and icons to buffs, but cannot come up with an easy way to represent them in the UI then, so will provide names in my examples, but you can think about it.

I think that buff schema can have "effect", "conditions" and "restrictions" props, among others. But maybe it's worth separating effects into another list and referencing them by id/code.

Examples:

Power 1: x1.1 to spell power

Power 2: x1.2 to spell power

Power 3, 4, 5

Armor 1: x0.9 to received damage

Armor 2, 3, 4, 5

Vitality 1: +10 total HP

Vitality 2, 3, 4, 5

Lifesteal 1: you restore 0.2 HP per each damage point

Lifesteal 2, 3

Payback 1: +0.1 power per each damage taken in the prev turn

Payback 2, 3, 4

Rage 1: x1.2 power if your you lost half your HP

Rage 2, 3, 4

Momentum 1: +0.1 power per damage dealt to enemy in prev turn

Momentum 2,3,4,5

Mythic power: x2 power, restriction: you can only use sealing words with letter a;

Mythic disruption: 30% chance of enemy missing a turn, restriction: you cannot use sealing words with letter e;

You can suggest 1-2 additional buff ideas

We also need to think of negative effects dealt by enemies

Then let's move on to moves

This is a very clean direction. Moving to a generalized `Actor` model and a modular `BuffSystem` makes the "Game Engine" approach much more viable.

Here is the Data Specification for **Bufs** and **Moves**. I have designed this to be "flat" enough to be readable in JSON, but expressive enough to handle the complex logic (conditions, reactions, and restrictions) you described.

Core Philosophy: The "Modifier" Pattern

Instead of hardcoding logic like "Lifesteal" or "Rage" into the game engine, we treat everything as a **Modifier**.

- **Stats:** HP, Power, Defense are just numbers we modify.
 - **Rules:** Sealing constraints are just lists of banned/required characters.
 - **Flow:** Turn skipping or extra actions are event flags.
-

1. The Buff/Effect Schema

Since you want to avoid assets (icons/names) where possible, this schema focuses on **generating descriptions from logic**.

TypeScript Definition

TypeScript

```
type TargetStat =
  | "damage_outgoing" // Power
  | "damage_incoming" // Protection/Armor
  | "max_hp"           // Vitality
  | "lifesteal"         // % of dmg returned as HP
  | "seal_banned_char" // Restriction: cannot use specific char
  | "enemy_skip_turn"  // Chance for enemy to miss turn
  | "chips_per_turn";  // Hand size

type ScalingSource =
  | "constant" // Fixed value
  | "hp_lost_percent" // For "Rage"
  | "dmg_taken_prev" // For "Payback"
  | "dmg_dealt_prev"; // For "Momentum"

interface BuffCondition {
```

```

    type: "hp_below_percent" | "turn_count_gt";
    value: number;
}

interface Modifier {
    target: TargetStat;
    operator: "add" | "multiply" | "append" | "override";
    baseValue: number | string; // 10, 0.2, "a"

    // Advanced Logic
    scaling?: {
        source: ScalingSource;
        factor: number; // e.g., 0.1 (10% of source)
    };
    condition?: BuffCondition;
}

interface GameBuff {
    id: string;
    rarity: "common" | "rare" | "mythic";
    isDebuff?: boolean; // If true, displayed in red

    // Optional: If you don't provide a name, UI generates one like "Damage
x1.2"
    label?: string;

    // The meat of the logic
    modifiers: Modifier[];
}

```

JSON Configuration Examples

Here is how your specific requests translate into this schema.

JSON

```

[
  {
    "id": "power_2",
    "label": "Power II",
    "rarity": "common",
    "modifiers": [
      { "target": "damage_outgoing", "operator": "multiply", "baseValue": 1.2
    }
  ],
  {
    "id": "armor_1",
    "label": "Iron Skin",
    "rarity": "common",
    "modifiers": [
      { "target": "damage_incoming", "operator": "multiply", "baseValue": 0.9
    }
  ],
  {

```

```

    "id": "lifesteal_1",
    "label": "Vampirism I",
    "rarity": "rare",
    "modifiers": [
      { "target": "lifesteal", "operator": "add", "baseValue": 0.2 }
    ]
  },
  {
    "id": "payback_1",
    "label": "Karma",
    "rarity": "rare",
    "modifiers": [
      {
        "target": "damage_outgoing",
        "operator": "add",
        "baseValue": 0,
        "scaling": { "source": "dmg_taken_prev", "factor": 0.1 }
      }
    ]
  },
  {
    "id": "rage_1",
    "label": "Berserk",
    "rarity": "rare",
    "modifiers": [
      {
        "target": "damage_outgoing",
        "operator": "multiply",
        "baseValue": 1.2,
        "condition": { "type": "hp_below_percent", "value": 0.5 }
      }
    ]
  },
  {
    "id": "mythic_disruption",
    "label": "Cursed Time",
    "rarity": "mythic",
    "modifiers": [
      // Benefit: 30% chance enemy skips
      { "target": "enemy_skip_turn", "operator": "add", "baseValue": 0.3 },
      // Restriction: Banned letter 'e'
      { "target": "seal_banned_char", "operator": "append", "baseValue": "e" }
    ]
  }
]

```

Additional Buff Ideas (Per your request)

1. **"Glass Cannon" (Mythic):** damage_outgoing x2.0, but max_hp x0.5. (High risk constant play).
2. **"Verbose Mage":** damage_outgoing x1.3, Restriction: Sealing word must be >5 letters. (Forces longer words).

2. The Move Schema (Actor Actions)

Moves need to communicate **Intent** (what's about to happen) and **Execution** (what actually happens).

TypeScript Definition

TypeScript

```
type MoveType =
  | "attack"
  | "buff_self"
  | "debuff_player"
  | "wait";

interface ActorMove {
  id: string;
  type: MoveType;

  // Display text for Intent (e.g. "Preparing to strike")
  intentLabel: string;

  // The actual numbers
  value?: number;

  // If the move applies a status effect (Buff/Debuff)
  appliesEffectId?: string;
}

interface Actor {
  id: string;
  hostile: boolean; // true
  name: string;
  hp: number;

  // Visuals (can be null for MVP)
  avatar?: string;

  // The logic
  moveList: ActorMove[]; // Cycle or Random pick from this list
  buffPool: string[]; // IDs of buffs they might drop

  // Dialogue (as discussed previously)
  dialogue?: ActorDialogue;
}
```

JSON Configuration Example (Actor + Moves)

JSON

```
{
  "id": "actor_forest_goblin",
```

```

"name": "Greedy Goblin",
"hostile": true,
"hp": 40,
"buffPool": ["lifesteal_1", "power_1", "payback_1"],
"moveList": [
  {
    "id": "goblin_stab",
    "type": "attack",
    "intentLabel": "is sharpening his knife",
    "value": 5
  },
  {
    "id": "goblin_spit",
    "type": "debuff_player",
    "intentLabel": "looks ready to spit",
    "appliesEffectId": "debuff_blindness_1" // defined in buffs.json
  },
  {
    "id": "goblin_steal",
    "type": "attack",
    "intentLabel": "reaches for your pockets",
    "value": 2,
    // Special logic: could trigger a 'steal_chip' event in engine
  }
]
}

```

3. Handling "Negative Effects" (Debuffs)

You mentioned needing to think about negative effects. In this system, they are just **Buffs** with a negative flag.

Example: "Blindness" (Debuff)

- **Context:** Player gets this if the Goblin spits on them.
- **JSON:**

JSON

```

{
  "id": "debuff_blindness_1",
  "label": "Blinded",
  "isDebuff": true,
  "modifiers": [
    // Makes Sealing harder?
    // Or reduces damage?
    { "target": "damage_outgoing", "operator": "multiply",
      "baseValue": 0.7 }
  ]
}

```

4. UI Representation Strategy

Since you want to keep it text-based/minimal:

- **Buff List:** A simple row of pills/tags below the player HP.
 - `[Power II] [Karmic] [Blinded]`
- **Tooltip:** Hovering the tag generates the description on the fly.
 - *Algorithm:* Iterate modifiers -> Format String.
 - *Example:* `Power II` -> "Multiplies Outgoing Damage by 1.2".
 - *Example:* `Karma` -> "Adds 10% of last turn's received damage to Attack".

This structure allows you to balance the game purely by editing a JSON file, satisfying your "Game Editor" requirement perfectly.

Ok, thanks! I'm a bit unsure about the fact that we abandon "effect" term (very common in fames) and only use "buffs" (which can also be debuffs), but your "modifiers" approach seems to account for that, and it agrees with the light-weight concept. Let's roll with it for now.

Small additional idea: new buff family - "Scale": x1.1 (or 1.2 etc.) power PER FRAGMENT used in a spell.

Additional mechanic: spell complexity limitation: you can use only max 3 level spells at the beginning. At some point you unlock a buff which removes this limit (should be a guaranteed buff, e.g. in the loot of the second chapter boss).

Now, let's start finalizing our work by creating final documents. Think back to what we discussed so far, and summarize them in these docs. I think we can have 3 docs:

1. `game-design.md` - describes game genre, key elements, battle mechanics, buff mechanics, progression, short description of potential setting etc. Does not include architecture details or JSON schema. This doc will help project manager to plan features and tasks for developers, and testers to test the game;
2. `data-schema-draft.md` - describes preliminary TS types for the main game entities. Should be design lean, smart and extensive. Should be considered a draft and can be changed during implementation. This will help developers build the core game logic;
3. `architecture.md` - describes folder structure for the future project, technical decisions and recommended technologies to use. This will help project managers and developers to keep the project consistent.

IMPORTANT: so far when designing your documents, you created them for me, a person who has the context of the game. When creating the final docs, please write them for people without the general context, don't assume that some information is known already (but you can cross-reference the docs). The 3 docs should be complete

info that I can delegate to other people to start building the project without additional comments from my side.

Let's start with a draft of the game-design.md ?

Here is the **Game Design Document** draft. It is written as a standalone specification for a development team.

Game Design Document: Spell Caster Quest

Version: 1.0 **Genre:** Strategic Word-Puzzle RPG **Platform:** Web (Mobile-responsive) **Target Audience:** Casual strategy players; fans of word games (e.g., *Wordle*) and lightweight RPGs.

1. Executive Summary

Spell Caster Quest is a turn-based battle game where players cast spells by combining word fragments into English words. The core loop revolves on a risk/reward mechanic: players must draft "Fragments" (English words) from a pool to build a "Spell," but the spell is only successfully cast if the player can validate it with a "Sealing Word" that linguistically connects all fragments.

The game combines the intellectual satisfaction of word puzzles with the tactical progression of a roguelite. It features a minimalist, modern aesthetic and a lighthearted tone.

2. Terminology & Definitions

- **Actor:** Any entity in the battle. The player is the protagonist; the enemy is a hostile Actor.
- **Fragment:** A game piece representing a single English word (e.g., "APPLE", "RUN").
- **Spell Formation:** The active area where players drag Fragments to construct a spell.
- **The Seal:** A user-typed word that validates the Spell Formation. It must satisfy specific linguistic rules connecting the Fragments.

- **Mystery Fragment:** A high-power Fragment with hidden letters (e.g., "??ST??") that reveals its true word only after being committed to the Spell Formation.
 - **Backfire:** A penalty mechanic. If a player commits Fragments to the Formation but fails to Seal them, they take damage.
 - **Manifestation:** The tactical effect chosen by the player *after* successfully Sealing a spell (e.g., Attack vs. Heal).
 - **Buff:** A persistent status effect (positive or negative) applied to an Actor.
-

3. Core Gameplay Loop (The Battle)

Battles are turn-based (Player Turn -> Enemy Turn).

3.1. Player Turn Phases

1. **Drafting:**
 - The player is presented with a **Pool** of random Fragments (approx. 10).
 - Some Fragments may be **Mystery Fragments** (marked with ? or obscured text).
 - The player drags desired Fragments into the **Spell Formation** area.
 - **Constraint:** Initially, the player is limited to a maximum spell size (e.g., 3 Fragments). This limit is removed via progression in later chapters.
2. **Commitment & Risk:**
 - Once a Fragment is placed in the Formation, it is "active."
 - If a **Mystery Fragment** is placed, it is immediately revealed (e.g., ??ST?? becomes GHOST).
 - **The Point of No Return:** Revealed Mystery Fragments cannot be removed from the Formation. The player must use them or clear the entire spell and suffer a **Backfire**.
3. **Sealing (The Puzzle):**
 - The player must type a valid English word (The Seal) into an input field.
 - **Validation Rule:** The Sealing Word must contain a specific linguistic connection to every Fragment in the Formation.
 - *Standard Rule:* The Sealing Word must contain a 2-letter substring (bigram) found in the Fragment.
 - *Example:* Fragment "APPLE" (contains 'PP', 'PL', 'LE') -> Sealing Word "SUPPLE" is valid.
 - **Success:** The spell is cast. Proceed to Manifestation.
 - **Failure:** If the player cannot find a word, they must "Clear" the spell.
 - **Backfire Penalty:** The player takes damage proportional to the number of Fragments in the failed spell (e.g., 5% Max HP per Fragment).

4. **Manifestation (The Execution):** Upon a successful Seal, the player chooses one of three effects to apply:
 - **Blast:** Deals 100% of the calculated spell power as direct damage.
 - **Focus:** Deals 50% damage and grants a stacking "**Sharpness**" Buff (increased damage for future turns).
 - **Drain:** Deals 30% damage and grants a stacking "**Mending**" Buff (passive HP regeneration).

3.2. Enemy Turn Phase

- The hostile Actor executes a move from their distinct moveset.
 - Moves can be:
 - **Attack:** Deal direct damage.
 - **Debuff:** Apply a negative Buff to the player (e.g., "Blindness").
 - **Wait/Taunt:** No action, often accompanied by dialogue.
-

4. Progression & Meta-Game

4.1. Structure

- **Chapters:** The game is divided into linear Chapters (Chapter 1 -> Chapter 2 -> ...).
- **Encounters:** Each Chapter contains multiple Hostile Actors.
 - Players can choose to fight "Standard" Actors to grind XP/Buffs.
 - Players must defeat the **Chapter Boss** to advance to the next Chapter.
 - This structure allows risk management: a confident player can rush the Boss; a cautious player can farm standard enemies for power.

4.2. Rewards (Loot)

- Winning a battle presents the player with a choice of **Bufs**.
- Each Actor has a specific "Loot Pool" of Buff IDs. The game randomly selects 3 from this pool for the player to choose from.
- **Buff System:**
 - Bufs are the primary way to "Level Up."
 - They are stackable and persistent.
 - *Types:*
 - **Stat Modifiers:** e.g., "Power I" (x1.1 Damage), "Vitality" (+10 HP).
 - **Scaling Modifiers:** e.g., "Wordsmith" (+10% Power *per Fragment* in the spell).

- **Restrictions (Risk/Reward):** High-power buffs that add constraints (e.g., "Double Damage, but you cannot use the letter 'E' in Sealing words").
- **Unlockable:** The "Spell Complexity Limit" removal is a guaranteed Buff drop from the Chapter 1 Boss (or similar milestone).

4.3. Game Over

- **Player HP <= 0:** The run ends.
 - *MVP:* Player restarts the current Chapter.
 - *Future:* Roguelite reset to Chapter 1.
-

5. UI & UX Direction

- **Visual Style:** Modern, clean, flat design (e.g., Vercel/Shadcn UI aesthetic).
 - **Battle Interface:**
 - Top: Enemy Avatar (simple/abstract) + Status Bar.
 - Center: Player Stats + Active Buffs (displayed as text tags/pills).
 - Bottom: Spell Formation Area + Fragment Pool + Input Field.
 - **Feedback:**
 - **Enemy Personality:** Enemies have unique "barks" (toast notifications or speech bubbles) for Greeting, Winning, and Losing.
 - **Visual Clarity:** Clear distinction between "Active" Fragments (in Formation) and "Inactive" (in Pool).
-

6. Content Requirements (Initial Scope)

- **Chapter 1:** 3 Standard Enemies, 1 Boss.
 - **Buff Library:** ~10 distinct buffs (Attack, Defense, Regen, Scaling).
 - **Word Dictionary:** Standard English dictionary (~10k-20k common words) for validation.
 - **Sealing Rules:** Default bigram matching.
-

End of Game Design Document

Thanks! before we proceed: I had some thoughts about the battle mechanics (spell+sealing) and came up with an alternative approach:

- Spell is a valid sentence of a specific structure, not just random words: "unleash fiery power", "cast huge mighty fireball";
- In the pool, you see words of different types (color-coded?): starter/command (verb) with certain base power; object (noun) with a modifier (e.g.x2); modifier (adjective) with additional modifier;
- Each spell requires strictly 1 command and 1 object; modifiers are optional and unlimited;
- each fragment (command, object, mod) has main attribute (base power, modifier) and maybe optional extra attribute (visible but unknown, revealed upon casting). On first levels - no extra attributes;
- sealing rules: can stay the same, but i think about having a simpler version, at least on the first levels - to be easier understandable. E.g.: each word has a letter marked in bold; the sealing word should contain all letters from the spells in the same order. E.g. *u*nleash f*i*ery powe*r* - seal should contain U → I → R, e.g. "universal". Possible variations in advanced levels - more than one letter; letter chosen at random after committing the word etc.
- Example process for an early level: pool of ~5 words, e.g. "unleash", "conjure", "curse", "fireball", "strike". Initially, only commands (unleash, conjure) are active, other are grayed out. You choose e.g. "conjure" - it moves to a staging/building area, becomes locked/greyed out - hint "object needed" on hover. Objects (last 3 words) become active. You choose "fireball", it goes to the build area but also locked and does not connect to "conjure" - hint "seal needed" on hover. "Seal" button appears. On click, sealing mode turns on, you see the selected fragments (*c*conjure, firebal*I*) and instruction: seal with any word with the letters in order: C → L. You enter "collection". The fragments connect into a spell card: "conjure fireball" with calculated total power with a visual effect on appearing. Your buffs are also taken into calculation. Then the damage is applied to the enemy. On further levels, modifiers and more complex spells are introduced.

Please tell me what you think about the updated mechanic. I think it might be more logical and clear to starting players. But I think the fragment variability might need additional think through. Commands can have base damage, e.g. displayed by a "⚡" icon with a number; but what should modifiers and objects have? Just variations of "x2", "x3" etc.? I think something more interesting can be introduced, but it should be simple (no complex lore or extra mechanics, strive for light-weight game system) and easily displayable (e.g. icon with number or some kind of badge) and readable. Maybe protection points?

Terminology - also validate, suggest improvements if you see some
This is a **major and positive** shift. I strongly agree with this direction.

Why this is better:

1. **Intuitive Fantasy:** "Constructing a sentence" (*Unleash Fiery Blast*) feels much more like being a wizard than connecting "Apple" to "Table."

2. **Clearer Logic:** The Syntactic structure (Verb → Adjective → Noun) provides a natural tutorial. Players intuitively know an adjective modifies a noun.
3. **Visual Parsing:** Color-coding parts of speech (Red Verbs, Blue Nouns, Green Adjectives) makes the board readable at a glance.

Here is a breakdown of how to implement this "Sentence System" while solving the **Variability** and **Terminology** questions.

1. The Structure (Syntax)

We treat the Spell Formation as a **Sentence Slot System**.

- **Slot 1: The Invocation (Verb) - Mandatory**
 - *Role:* The Engine. It provides the **Base Power**.
 - *Icon:* ⚡ (Energy/Power).
 - *Examples:* Cast (3), Unleash (5), Invoke (4).
- **Slot 2: The Sigil (Adjective) - Optional (Unlimited)**
 - *Role:* The Booster. It scales the previous value.
 - *Icon:* x or +.
 - *Examples:* Mighty (x2), Dark (+3), Quick (+1).
- **Slot 3: The Focus (Noun) - Mandatory**
 - *Role:* The Payload. It determines **What happens** (Damage, Block, or Heal) and adds a flat bonus.
 - *Icon:* Changes based on type (⚔, 🛡, ❤).
 - *Examples:* Fireball (⚔ 2), Barrier (🛡 4), Spirit (❤ 2).

Example Spell:

Unleash (⚡5) + Mighty (x2) + Fireball (⚔ 2)

Calculation: $(5 * 2) + 2 = 12$ **Damage.**

This solves the "Variability" problem:

- **Verbs** are always raw numbers.
 - **Adjectives** are always math operators.
 - **Nouns** determine the **Effect Type**. This is your meaningful variability. You don't need complex lore; just standard RPG icons.
-

2. The Sealing Mechanic (Revised)

Your idea of the **Subsequence Rule** is perfect. It is easier to visualize than "Bigrams" but still hard to solve with a restricted vocabulary.

The Rule: Every Fragment has **one** highlighted letter (The "Rune").

- cast
- Mighty
- Fireball

The Task: Type a word that contains C, T, I **in that order**.

- *Input:* "Creation" -> **Creation**. (Matches C, T, I? Yes).
- *Input:* "Citrus" -> Matches C, I, T... (Wrong order).

Progression:

- *Level 1:* Highlight is always the **first letter** (Easier). cast Fireball -> "Coffee".
- *Level 2+:* Highlight is random. Cast Fireball -> S...L -> "Soul".

3. Updated Terminology

Let's keep it "Magical" but descriptive.

Concept	New Term	Description
Verb	Invocation	The starting command.
Adjective	Sigil	A modifier rune.
Noun	Focus	The object target of the spell.
Staging Area	The Scroll	Where the sentence is built.
Sealing	Binding	Connecting the runes (letters) to cast.

4. Gameplay Flow (Walkthrough)

Phase 1: The Draft The Player sees the **Pool**.

- **Active:** `Invoke` (⚡3), `Cast` (⚡2) (Highlighted).
- **Inactive:** `Fireball` (✖), `Shield` (🛡), `Greater` (x2).

Phase 2: The Build

1. Player clicks `Invoke`.
 - `Invoke` moves to **The Scroll**.
 - Verbs in Pool become inactive.
 - **Sigils** and **Focuses** become active.
2. Player clicks `Greater`.
 - `Greater` moves to Scroll after `Invoke`.
3. Player clicks `Fireball`.
 - `Fireball` moves to Scroll.
 - The Sentence is complete: **Invoke Greater Fireball**.
 - **"BIND SPELL"** button appears.

Phase 3: The Binding (Sealing)

1. Battle interface dims. Focus is on the Scroll.
2. The "Runes" (target letters) light up:
 - `Invoke` -> **V**
 - `Greater` -> **E**
 - `Fireball` -> **L**
3. Prompt: *"Bind: $V \rightarrow E \rightarrow L$ "*
4. Player types: **"NOVEL"**.
5. Success! Spell Casts.

5. Impact on "Manifestations"

Design Note: Since the **Focus (Noun)** now determines if the spell is an Attack (Fireball), Defense (Barrier), or Heal (Spirit), we **no longer need** the post-spell menu (Blast/Focus/Drain) we discussed previously.

- **Benefit:** Faster gameplay loop. The strategy happens *during* drafting, not after.
- **Risk:** If the RNG only gives you Defensive Nouns, you can't attack.
- **Fix:** Ensure the "Deck" (Pool) always guarantees at least 1 Attack Noun.

6. Architecture Note

For the `data-schema-draft.md`, this simplifies things. We don't need complex "movesets" for players. We just need a **Fragment Database**.

TypeScript

```
type FragmentType = "invocation" | "sigil" | "focus";

interface Fragment {
  id: string;
  word: string; // "Fireball"
  type: FragmentType;

  // Visuals
  runeIndex: number; // Which char is bold? 0 = first letter.

  // Stats
  basePower?: number; // For Invocations (5)
  multiplier?: number; // For Sigils (2)
  effectType?: "damage" | "block" | "heal"; // For Focus
  flatBonus?: number; // For Focus (+2)
}
```

Does this "Sentence System" (Invocation -> Sigil -> Focus) feel like the final iteration for the mechanics? If so, I am ready to generate the final documentation.

Thanks! I like how you updated the system with "focus" providing "payload"/what happens. Now it's clear distinction between the three. What I'm not sure is that it's not clear what the modifier (sigil) applies to - the power level from invocation or the number from the focus (e.g. ✂ 2, 🛡 4)? In your example, it's the power level, but that doesn't feel very intuitive for me. Would you argue with that? Possible mitigation: focus provides only type (✂, 🛡, ❤); optionally it can have secondary "boost" (+2, +5 etc.). Another concern - your system limits a spell to only one effect (only attack, only block). Maybe we could fix it with some modifiers having additional bonuses (e.g. x2 +2 🛡 - you can use in an attack spell but also add some protection along the way). Also, do you think we should add any more aspects aside from ✂, 🛡, ❤? Without complicating the understanding?

Please let me know what you think. I think we are almost ready to produce a revised game design document. Please rewrite only the affected parts of your previous version of GDD to incorporate the new spell mechanic. Additional task: think back through our conversation and detect what did we miss/forget to include in the GDD? I mean, aside from the info that will go into data schema and code architecture docs. Come up with 3-4 points if you can

This is a crucial refinement. Ambiguity in math is the enemy of strategy games. If players can't calculate the outcome in their heads instantly, they will feel frustrated.

1. Resolving the Math Ambiguity

You are right: $(5 * 2) + 2$ is confusing. Why multiply the verb but add the noun?

The Solution: Left-to-Right "Energy Flow" Think of the spell as a pipeline.

1. **Invocation (Source):** Generates raw **Energy** points. (e.g., `Unleash` = 5 Energy).
2. **Sigil (Pipe/Valve):** Modifies the flowing Energy. (e.g., `Mighty` = Multiplies Energy by 2).
3. **Focus (Outlet):** Converts the final Energy into a specific **Effect Type**. (e.g., `Fireball` = Converts Energy to Damage).

New Formula: (Invocation Base * Sigil Multipliers) = Total Potency applied as Focus Type.

- *Example:* `Unleash` (5) + `Mighty` (x2) + `Fireball` (🔪)
- *Math:* .
- *Result:* **10 Damage.**

What about the Focus Bonus? If a Focus has a number (e.g., `Barrier` +2), it simply adds to the final total.

- *Example:* `Cast` (3) + `Barrier` (+5) -> **8 Block.**

2. Solving Single-Effect Limits (Hybrid Spells)

You are absolutely correct that "Only Attack" or "Only Block" is too limiting. **Solution: Sigil Side-Effects.** We keep the Focus as the *Main Type* (determining the primary color/icon of the result), but allow Sigils to add secondary stats.

- **Vampiric Sigil:** x1.0 Power but adds `Lifesteal` trait.
- **Guarded Sigil:** x1.0 Power but adds +3 Block.
- *Result:* `Unleash` (5) + `Guarded` (+3 🛡️) + `Fireball` (🔪) = **5 Damage + 3 Block.**

3. Additional Aspects (Beyond 🔪 🛡️ ❤️)

We should add **one** more core aspect to handle "Utility/Speed," but keep it rare.

- **Aether (🌀):** Represents "Mana" or "Flow."
 - *Effect:* Generates "Next Turn Energy" or draws extra chips.
 - *Why:* Allows "setup turns" where you don't attack or block, but prepare a massive next turn.

Revised GDD Sections

Here are the rewritten sections for the Game Design Document, incorporating the "Sentence Construction" mechanics.

3. Core Gameplay (Revised)

3.1. Spell Construction (The Sentence System) Instead of piling random words, players construct a magical sentence following a strict syntactic structure: **Invocation Sigil(s) Focus**.

- **1. Invocation (The Source - Mandatory):**
 - *Role:* Sets the base **Energy** value of the spell.
 - *Type:* Verb.
 - *UI:* Icon ⚡ + Number.
 - *Examples:* Cast (3), Channel (4), Unleash (6).
- **2. Sigil (The Modifier - Optional, Unlimited):**
 - *Role:* Modifies the Energy flowing from the Invocation. Can also add secondary side-effects (e.g., small Block amount).
 - *Type:* Adjective.
 - *UI:* Operator (x2, +1) or Status Icon.
 - *Examples:* Mighty (x2), Quick (+1 Energy), Guarded (+3 Block).
- **3. Focus (The Outlet - Mandatory):**
 - *Role:* Determines the **Primary Effect** (Damage, Block, Heal) that the final Energy is converted into.
 - *Type:* Noun.
 - *UI:* Type Icon (⚔/🛡/❤).
 - *Examples:* Fireball (⚔), Barrier (🛡), Spirit (❤).

Calculation Logic:

The Result is applied as the **Focus Type**.

3.2. The Binding (Sealing Mechanic) Once a valid sentence is constructed (e.g., Unleash + Mighty + Fireball), the player must **Bind** the spell to cast it.

- **The Rule:** Every Fragment has one "Rune" (a specific letter) highlighted.
 - **The Puzzle:** The player must type a single English word that contains the Runes of all active Fragments **in the correct order**.
 - *Example:* unleash mighty Fireball.
 - *Target:* **U ... M ... L**.
 - *Valid Input:* "UMBRELLA".
 - **Progression:**
 - *Early Game:* The Rune is always the **first letter** of the fragment (Easier to scan).
 - *Late Game:* The Rune position is randomized or hidden until the fragment is drafted.
-

Missing GDD Points (Crucial Additions)

Here are 4 points we haven't explicitly documented yet but are vital for the game to function:

1. The "Deck" Lifecycle (Replenishment):

- *Question:* When I use "Fireball", is it gone forever? Does it shuffle back in?
- *Proposal: Infinite Bag with Rarity.* We don't simulate a decreasing "Deck" (like Slay the Spire). Instead, every turn, the Pool is refilled from a random generator based on unlocked rarities.
- *Why:* Matches the "Scrabble" vibe—you just get new letters. It simplifies coding (no discard pile logic).

2. Enemy Intent System:

- *Question:* How does the player know to use a "Shield" spell?
- *Proposal: Telegraphed Intent.* The Enemy displays their next move *above their head* at the start of the Player's turn (e.g., "X 12" or "z^z Buffing").
- *Why:* Without this, Defense spells are pure guessing, which frustrates strategy players.

3. Fragment "Cost" or Limits:

- *Question:* What stops me from using 10 Sigils in one turn?
- *Proposal: Soft Limit via Difficulty.* You *can* use 10 Sigils, but finding a Sealing Word with 12 specific letters in order (U..M..L..X..Y..Z...) is linguistically impossible or extremely hard.
- *Balance:* The "Binding Puzzle" *is* the resource cost.

4. Tutorial / Onboarding:

- *Question:* How do they learn the U..M..L rule?
- *Proposal: The "Ghost" Hand.* The first battle is scripted. The game highlights the exact chips to click and *shows* the required letters pulsing, possibly filling the input box automatically as a demo ("Type 'UMBRELLA' to bind").

Shall we move to generating the **Data Schema Draft** (TS Interfaces) now, incorporating these mechanics?

Quick question before we proceed: please brainstorm other naming options for invocation, sigil, focus

Here are a few thematic "Flavor Packs" for naming the spell components.

The goal is to find names that hint at the **mathematical function** (Base Number Multiplier Effect Type) while keeping the **fantasy vibe**.

Option 1: The "Arcane" Set (Mystic & Classic)

Focuses on the idea of writing/crafting magic.

- **Invocation Script** (or **Scroll**)
 - *Why:* Short, implies the "starting text" or foundation.
- **Sigil Rune**
 - *Why:* "Rune" is universally understood as a magical symbol that adds power or quality.
- **Focus Form**
 - *Why:* The raw magic takes a specific physical "Form" (Fire, Shield, etc.).
- *Example:* "Script" + "Rune" + "Form" = **Spell**.

Option 2: The "Kinetic" Set (Energy & Flow)

Focuses on the flow of power from source to output.

- **Invocation Spark**
 - *Why:* It's the ignition that creates the initial energy value.
- **Sigil Flux**
 - *Why:* Sounds volatile and additive; implies changing/bending the energy.
- **Focus Core**
 - *Why:* The dense center of the spell that determines what it actually is.
- *Example:* "Spark" + "Flux" + "Core" = **Burst**.

Option 3: The "Ritual" Set (Spoken Magic)

Focuses on the idea of a wizard speaking words of power.

- **Invocation Chant**
 - *Why:* You start by chanting. It sets the volume/power.
- **Sigil Accent** (or **Tone**)
 - *Why:* It modifies *how* the chant is spoken.
- **Focus Echo**
 - *Why:* The result that reverberates into the world.
- *Example:* "Chant" + "Accent" + "Echo".

Option 4: The "Component" Set (Tactile/Item-based)

Focuses on the chips looking like physical objects.

- **Invocation Source**
 - *Why:* The battery/origin of the power.
- **Sigil Charm**
 - *Why:* A small trinket you add for luck or bonuses.
- **Focus Relic**

- *Why:* The main artifact you are activating.

My Recommendation?

I prefer **Option 1 (Script / Rune / Form)** or a mix:

1. **Cast** (Verb) - *Simple, action-oriented.*
2. **Rune** (Adjective) - *Short, readable, fits "modifier".*
3. **Orb** (Noun) - *Visual, easy to iconize (Spheres of Fire/Ice/Shield).*

So you build a spell: **Cast** (5) **Mighty Rune** (x2) **Fire Orb** (x).

Which direction appeals to you most? Or shall we stick to the original Invocation/Sigil/Focus?

How about "command" - "aspect" - "entity"? Less arcane but i think better graspable.

Please tell me what you think, and also come up with the updated version of the GDD. Just as a reminder - this doc is not for me, don't assume any previous knowledge; make sure to include all discussed game elements , mechanics, preogression, meta; include discussed visual aspects, potential setting; technical aspects , but no actual code or data schema

"Command - Aspect - Entity" works very well. It sounds structured and logical, like a programming syntax for magic, which fits the "constructing" theme perfectly.

- **Command:** (Verb) The trigger. High energy.
- **Aspect:** (Adjective) The quality. Modifiers.
- **Entity:** (Noun) The manifestation. The physical result.

Here is the finalized **Game Design Document**. It is written as a formal specification for your development team.

Game Design Document: Spell Caster Quest

Version: 1.0 **Genre:** Strategic Word-Puzzle RPG **Platform:** Web (Mobile-Responsive) / Future: Electron **Tech Stack:** React, Vite, Tailwind, LocalStorage (MVP)

1. Executive Summary

Spell Caster Quest is a turn-based battle game where players cast spells by constructing valid English sentences. Unlike traditional RPGs where skills are cooldown-based, here magic is "syntactic": players must draft words (**Commands**, **Aspects**, and **Entities**) to define the spell's power and effect.

The core challenge is a hybrid of strategy and word puzzle: the more complex the constructed sentence, the harder it becomes to "**Seal**" (cast) it. Players must find a single English word that linguistically connects all the chosen fragments. Failure to seal a constructed spell results in a **Backfire**, damaging the player.

Key Hook: "Write your magic. Bind it with words. Risk it all for power."

2. Terminology & Definitions

- **Actor:** Any character in battle (Player or Enemy).
 - **Fragment:** A playable game piece representing a word (e.g., "STRIKE", "QUICK").
 - **The Scroll:** The active staging area where the player builds their spell sentence.
 - **The Seal:** A player-typed word that validates the spell by matching specific letters from the Fragments.
 - **Binding:** The action of successfully typing a Seal to cast the spell.
 - **Backfire:** The penalty damage taken when a player abandons a spell or fails to Bind it.
 - **Buff:** A passive, persistent status effect (e.g., "+10% Power").
-

3. Core Gameplay Loop (The Battle)

Battles are 1v1 turn-based encounters.

3.1. Player Turn: The Spell Construction System

Instead of simple damage cards, spells are constructed sentences following a strict syntax: **Slot 1 (Mandatory) Slot 2 (Optional/Unlimited) Slot 3 (Mandatory)**.

The Components (Fragments)

1. **Command (Verb):**

- *Function:* The Engine. Sets the **Base Energy**.
 - *Examples:* CAST (3), INVOKE (5), UNLEASH (8).
 - *UI:* Flash/Energy Icon (⚡).
2. **Aspect (Adjective):**
- *Function:* The Modifier. Multiplies the energy or adds traits.
 - *Examples:* MIGHTY (x2), QUICK (+1 Energy), VAMPIRIC (Adds Lifesteal).
 - *UI:* Operator Icon (x, +).
3. **Entity (Noun):**
- *Function:* The Payload. Converts the final Energy into an Effect Type (Attack, Defense, Heal).
 - *Examples:* FIREBALL (🔪 Attack), BARRIER (🛡️ Block), POTION (❤️ Heal).
 - *UI:* Type Icon.

The Math

The result is applied as the Entity's Effect Type.

Example: Player selects: UNLEASH (5) + MIGHTY (x2) + FIREBALL (🔪) Result: **10 Damage**.

3.2. The Binding Mechanics (The Puzzle)

To cast the constructed sentence, the player must **Bind** it.

1. **The Rune Rule:** Every Fragment has one specific letter highlighted (The Rune).
 - *Level 1:* Always the **First Letter** (e.g., UNLEASH).
 - *Advanced:* Random letter (e.g., UNLEASH).
2. **The Constraint:** The player must type a valid English word that contains all Runes from the Scroll **in the exact order**.
 - *Scenario:* UNLEASH MIGHTY FIREBALL.
 - *Requirement:* A word containing **U ... M ... F**.
 - *Valid Input:* "UNCOMFORTABLE".
3. **Success:** The spell is cast.
4. **Failure:** If the player cannot find a word, they must clear the Scroll. This triggers a **Backfire** (Damage = 5% Max HP per Fragment cleared).

3.3. Enemy Turn & Intent

- **Telegraphed Intent:** At the start of the Player's turn, the Enemy's next move is displayed (e.g., "🔪 8" or "zZ Buffing"). This allows the player to react (e.g., building a Barrier Entity instead of a Fireball).
- **Enemy Moves:**
 - **Attack:** Deal damage.
 - **Debuff:** Apply negative effects (e.g., "Blindness" - hides Runes).

- **Disrupt:** Add "Garbage Fragments" to the player's pool.

3.4. The Deck Lifecycle

There is no "Draw Pile" or "Discard Pile."

- **Infinite Pool:** The pool is refilled to full (e.g., 7-10 chips) at the start of every turn.
 - **RNG Logic:** The refill is random but weighted to ensure playability (always guarantees at least 1 Command and 1 Entity).
-

4. Progression & Systems

4.1. Campaign Structure

- **Chapters:** Linear progression (Chapter 1 2 3).
- **Encounters:** Each chapter has a set of Enemies leading to a **Chapter Boss**.
- **The Gate:** Players can fight standard enemies to farm XP/Buffs but must defeat the Boss to advance to the next Chapter.

4.2. The Loot System (Buffs)

The primary way to grow stronger is collecting **Buffs** (Passive Effects) after winning battles.

- **Buff Selection:** Players choose 1 of 3 random Buffs from the defeated Enemy's specific loot pool.
- **Buff Archetypes:**
 - **Stat Boost:** "+10 Max HP".
 - **Scaling:** "+1 Base Power per Aspect used".
 - **Risk/Reward (Mythic):** "x2 Damage, but Sealing Words must not contain 'E'".
- **Progression Gate:** The player starts with a "Complexity Limit" (max 3 Fragments per spell). This limit is removed by a specific Boss Drop in Chapter 1.

4.3. Mystery Fragments

To add risk, some Fragments in the pool appear as **Hidden** (e.g., ??ST??).

- **Benefit:** High stats (e.g., Base 10 Command).
- **Risk:** The letters (and the Rune) are revealed only *after* dragging it to the Scroll.

- **Lock:** Once revealed, it cannot be removed without a Backfire.
-

5. Visual & UX Direction

5.1. Aesthetic

- **Style:** Minimalist, "Modern Web" aesthetic (Flat, clean typography, rounded corners). Think *Vercel* or *Shadcn/ui*.
- **Color Coding:**
 - **Command:** Yellow/Electric (⚡).
 - **Aspect:** Blue/Magic (✨).
 - **Entity:** Red (Attack), Green (Heal), Grey (Block).

5.2. Key UX Features

- **Drag & Drop:** Smooth, tactile feel for moving fragments.
 - **Validation Hints:**
 - Hovering a Fragment shows connection lines to valid slots.
 - The Input Field validates the Sealing Word in real-time (Green checkmark when the regex matches).
 - **Enemy Personality:** Simple "Barks" (Toast notifications) for Enemy greetings, win lines, and lose lines.
-

6. Technical Specifications (Summary)

6.1. Architecture

- **Frontend:** React + Vite.
- **State Management:** React Query (TanStack Query) for game state/persistence; Context API for ephemeral UI state (drag-and-drop).
- **Persistence:** LocalStorage (mapped via an Async Service Pattern to allow future backend migration).
- **Styling:** Tailwind CSS.

6.2. Configuration (The Engine)

The game content must be decoupled from the code. All game data is loaded from JSON files:

- `chapters.json`: World map and enemy lists.
- `actors.json`: Enemy stats, movesets, and dialogue.
- `fragments.json`: The dictionary of playable words (Command/Aspect/Entity) and their stats.
- `buffs.json`: Definitions of passive effects.

6.3. Dictionary Handling

- **Validation:** Use a compressed English dictionary (e.g., top 20k common words) to validate Sealing Words.
- **Performance:** Dictionary should be loaded once at startup.

End of Document